AI PREVENTIVE MAINTENANCE SYSTEM

[Author Name(s)]
Department of Electronics and Communication Engineering
Malla Reddy University, Hyderabad, India

Abstract— Industrial machines may develop faults due to abnormal vibration, imbalance, misalignment, looseness, and mechanical wear. This paper presents a low-cost ESP32-based AI Preventive Maintenance System prototype for real-time machine condition monitoring. An MPU6050 inertial sensor measures acceleration changes in a BO DC motor assembly. A potentiometer provides speed adjustment through an L298N motor driver, allowing the motor to be operated at different speeds while its vibration behaviour is observed. The ESP32 processes sensor readings and classifies the operating condition using predefined warning and critical thresholds. Machine information is presented on a 16×2 I2C LCD and a local web dashboard. A buzzer provides an audible alert during a critical condition, while recorded values can be downloaded in CSV format for later examination. The prototype demonstrates the integration of embedded sensing, motor control, local IoT monitoring, alert generation, and data logging. The present implementation uses threshold-based condition classification; trained machine-learning-based fault prediction and verified industrial performance evaluation are identified as future work.

Index Terms— Preventive Maintenance, ESP32, MPU6050, IoT, Vibration Monitoring, Motor Control, Web Dashboard, Data Logging.

I. INTRODUCTION

Industrial machines are expected to operate reliably for long periods. Rotating components such as motors can gradually develop imbalance, looseness, misalignment, bearing wear, or other mechanical problems. These conditions may change the vibration pattern of a machine before a complete failure occurs. If such changes are not noticed, they can contribute to unexpected stoppages, reduced productivity, and increased maintenance effort.

Traditional maintenance approaches commonly include corrective maintenance after a failure and preventive maintenance at fixed time intervals. Corrective maintenance can result in unplanned downtime, while fixed schedules may lead to servicing too early or too late. Condition monitoring provides an additional approach by observing machine parameters during operation and identifying abnormal behaviour. Vibration is a useful parameter for rotating equipment because mechanical changes can influence the acceleration measured on the machine body.

The proposed project uses an ESP32 microcontroller and an MPU6050 sensor to demonstrate vibration-based monitoring on a small BO DC motor. The motor speed can be varied using a potentiometer and an L298N driver. The ESP32 processes the sensor data and presents the machine condition through an LCD and a web dashboard. A buzzer is activated when the vibration indicator reaches a critical threshold. Monitoring values can also be downloaded as a CSV file for further analysis.

A. Objectives

The main objectives are: (i) to measure vibration-related acceleration changes using the MPU6050; (ii) to control the BO motor speed using a potentiometer and PWM; (iii) to process sensor readings on the ESP32; (iv) to classify the operating condition into NORMAL, WARNING, and CRITICAL states; (v) to display current information locally; (vi) to generate an alert for critical vibration; and (vii) to store readings in a format suitable for offline analysis.

B. Scope of the Work

The work is a low-cost educational prototype intended to demonstrate the basic stages of condition monitoring. It focuses on a single motor, local processing, threshold-based status classification, a local web interface, and CSV logging. It does not claim that a trained artificial-intelligence model is currently predicting a specific mechanical fault. Machine-learning-based classification, long-duration testing, and deployment on industrial machinery require additional data and validation.

II. RELATED WORK

Condition monitoring is widely used to assess the operating state of machinery. Vibration analysis is a common method for examining rotating equipment because changes in balance, alignment, mounting, and mechanical condition can affect measured vibration. The interpretation of vibration depends on sensor location, mounting stiffness, machine speed, measurement bandwidth, and the signal-processing method. Consequently, readings from a small prototype should be treated as indicators for demonstration rather than as certified industrial diagnostic measurements.

Microcontroller platforms have made it possible to build compact monitoring systems at relatively low cost. The ESP32 combines processing capability with wireless networking, making it suitable for local data acquisition and web-based visualization. The MPU6050 integrates a three-axis accelerometer and a three-axis gyroscope and communicates with a controller through I2C. In this project, acceleration readings are used to form a vibration indicator. The L298N motor driver provides the interface between low-power controller signals and the BO motor supply.

IoT-based monitoring systems commonly combine sensing, embedded processing, visualization, and data storage. A local dashboard can make current values available without requiring a dedicated industrial monitoring platform. CSV export provides a simple way to inspect time-stamped observations in spreadsheet software. These concepts form the basis of the proposed prototype.

III. PROPOSED SYSTEM ARCHITECTURE

The system consists of an ESP32 development board, MPU6050 sensor, potentiometer, L298N motor driver, BO DC motor, 16×2 I2C LCD, active buzzer, and a Wi-Fi-enabled web dashboard. The MPU6050 measures acceleration changes and transfers the readings to the ESP32 through the I2C bus. The potentiometer is connected to an analog input and is used to set the motor speed. The ESP32 converts the input into a PWM control signal for the L298N driver.

The controller calculates a vibration indicator from the acceleration components and compares it with configured limits. Based on the comparison, the system updates the machine status. The LCD provides a compact local indication, while the web dashboard presents additional parameters and vibration history. If the indicator crosses the critical limit, the buzzer is activated. The data-logging facility stores observations so that changes over time can be reviewed.

A. System Workflow

Machine operation → MPU6050 sensing → ESP32 acquisition and processing → vibration threshold comparison → machine status → LCD / web dashboard / buzzer → CSV data logging.

B. Vibration Monitoring

The MPU6050 provides acceleration measurements along three axes, represented as Ax, Ay, and Az. A resultant acceleration magnitude can be calculated as:

$$A = \sqrt{(Ax^2 + Ay^2 + Az^2)}$$

Changes in the resultant magnitude can be used to form a vibration indicator. The measured acceleration may include the effect of gravity, sensor orientation, motor mounting, and dynamic movement. For more meaningful vibration analysis, a practical implementation should establish a baseline, keep sensor orientation consistent, and apply suitable filtering or baseline compensation. The current prototype uses the calculated indicator and configured thresholds to demonstrate condition classification.

C. Motor Speed Control

The potentiometer provides an analog voltage to the ESP32. The controller reads this value and maps it to a PWM duty cycle. The PWM output is applied to the L298N motor driver, which controls the power delivered to the BO motor. This arrangement allows the motor to be operated at different speed settings while vibration readings are observed. The motor driver and motor must use suitable power supplies, and the controller ground must be connected appropriately to the driver ground.

D. Machine Condition Detection

The vibration indicator is compared with two configured limits. Values below the warning limit are classified as NORMAL. Values that reach the warning region are classified as WARNING, and values that reach or exceed the critical limit are classified as CRITICAL. The limits should be established through repeatable baseline measurements on the assembled prototype. They are application-specific and should not be treated as universal limits for all motors.

E. Alert and Monitoring

When a critical condition is detected, the ESP32 activates the buzzer. The LCD and web dashboard display the current status and relevant machine parameters. A clear separation between warning and critical indications helps the operator understand whether the reading requires attention. The buzzer logic should avoid unnecessary repeated triggering and should return to the defined inactive state when the measured condition no longer meets the alert criteria.

IV. HARDWARE REQUIREMENTS

The prototype uses the following hardware components:

Component	Purpose in the prototype
ESP32 development board	Main controller, sensor processing, Wi-Fi dashboard and data handling
MPU6050 sensor	Three-axis acceleration measurement for vibration indication
L298N motor driver	Motor drive interface controlled using PWM
BO DC motor	Rotating machine model used for demonstration
Potentiometer	User input for motor speed adjustment
16×2 I2C LCD	Local display of machine parameters and status
Active buzzer	Audible alert during critical vibration
Breadboard and jumper wires	Electrical interconnection during prototyping

The ESP32 acts as the central controller. The MPU6050 is mounted firmly on or near the motor assembly so that it can sense vibration consistently. Loose sensor mounting can introduce measurement errors. The I2C LCD shares the I2C communication bus with the sensor when the address and wiring permit. The L298N is used to drive the motor; the motor current must remain within the driver's practical operating limits, and the motor should be powered from an appropriate supply rather than from an ESP32 GPIO pin.

V. SOFTWARE DESIGN AND IMPLEMENTATION

A. Development Environment

The embedded firmware can be developed using the Arduino IDE with Embedded C/C++. The ESP32 firmware is responsible for initializing the peripherals, reading the MPU6050, acquiring the potentiometer input, generating PWM, updating the LCD, serving the web dashboard, controlling the buzzer, and recording monitoring data. The dashboard can be implemented using HTML, CSS, and JavaScript and served through the ESP32 over a local Wi-Fi connection.

B. Sensor Acquisition and Processing

During initialization, the controller configures the I2C interface and checks whether the MPU6050 is responding. Acceleration values are then read at a regular interval. The readings are converted into a vibration indicator and compared with the configured warning and critical limits. A consistent sampling interval is important when examining changes over time. If additional filtering is introduced, its effect on response time and the displayed values should be checked.

C. Web Dashboard

The ESP32 hosts a local web dashboard that provides a convenient view of the machine's operating information. The interface is intended to show temperature, humidity, motor speed, vibration, machine health status, and vibration history as described in the project implementation. The values shown depend on the sensors and firmware features actually connected and enabled in the final prototype. The dashboard can refresh the displayed readings periodically so that changes are visible during motor operation.

D. CSV Data Logging

CSV logging stores monitoring observations in a structured text format. A useful record may include a time or elapsed-time field, motor speed setting, acceleration components, calculated vibration indicator, and machine status. The downloaded file can be opened in spreadsheet software to plot trends, compare operating conditions, and identify periods in which the warning or critical state occurred. Logging intervals and file size should be selected so that data collection does not interfere with the real-time monitoring loop.

E. Firmware Organization

For easier debugging, the firmware can be organized into separate functions for sensor reading, vibration calculation, motor control, status classification, LCD updating, dashboard response, buzzer control, and CSV generation. Separating these tasks makes it easier to test each function independently. Error handling should cover sensor initialization failure, invalid readings, Wi-Fi disconnection, and unavailable display communication. A sensor error should be shown as an unavailable or fault state rather than being interpreted as a normal machine condition.

VI. ALGORITHM

The proposed monitoring sequence is as follows:

• Initialize the ESP32, I2C bus, LCD, MPU6050, PWM output, buzzer, and Wi-Fi connection.

• Read the potentiometer value and map it to the required motor PWM duty cycle.

• Send the PWM signal to the L298N motor driver to control the BO motor.

• Acquire the three-axis acceleration readings from the MPU6050.

• Calculate the resultant acceleration magnitude and derive the vibration indicator.

• Compare the indicator with the predefined warning and critical thresholds.

• Classify the machine state as NORMAL, WARNING, or CRITICAL.

• Update the LCD and the values presented on the local web dashboard.

• Activate the buzzer when the critical condition is detected.

• Record the monitoring data in CSV format and repeat the process.

VII. COMMUNICATION AND DATA FLOW

The MPU6050 communicates with the ESP32 through I2C, which carries sensor configuration and measurement data over shared clock and data lines. The I2C LCD uses the same type of interface, subject to compatible addresses and bus wiring. PWM and general-purpose input/output pins are used for motor speed control and buzzer activation. Wi-Fi provides the network connection for the local dashboard. These interfaces serve different roles: I2C is used for peripheral communication, PWM for motor control, GPIO for digital outputs, and Wi-Fi for dashboard access.

The monitoring sequence should be scheduled so that web requests or display updates do not cause long delays in sensor acquisition. A regular sampling interval makes the vibration history easier to interpret. If the firmware performs several operations in one loop, each operation should be kept short or scheduled so that a slow dashboard request does not prevent timely sensor updates.

VIII. SYSTEM INTEGRATION AND TESTING

Integration is best performed in stages. First, the ESP32 and MPU6050 should be tested independently to confirm that valid acceleration readings are received. Second, the potentiometer and L298N motor-control path should be checked at low motor speed before increasing the PWM duty cycle. Third, the LCD and buzzer should be tested using known status values. The web dashboard should then be checked on a device connected to the same local network. Finally, the complete system should be operated while confirming that sensing, motor control, status display, alert generation, and CSV logging work together.

The sensor should be firmly attached to the motor assembly and its position should remain unchanged during comparative tests. Readings can be collected with the motor stopped and at several speed settings to understand the baseline behaviour of the prototype. Controlled changes in mounting or balance may be used for demonstration only when performed safely. These tests can help select initial thresholds, but they do not establish a universal fault diagnosis capability.

A. Reliability Considerations

The motor must be mechanically secured, and exposed rotating parts should be guarded during testing. The motor supply should be appropriate for the motor and driver. The ESP32, sensor, and display should not be exposed to motor supply voltage. Wiring should be checked for correct polarity and common ground where required. Firmware should start with the buzzer inactive and should handle missing sensor data without generating misleading NORMAL status indications.

IX. RESULTS AND DISCUSSION

The prototype architecture demonstrates how a low-cost embedded controller can combine vibration sensing, adjustable motor speed, local status indication, web monitoring, and data logging. The LCD provides an immediate indication near the hardware, while the dashboard allows the user to observe additional information and vibration history. The three-level status approach makes the threshold decision understandable: NORMAL indicates that the current indicator is below the warning limit, WARNING indicates that the warning region has been reached, and CRITICAL indicates that the critical limit has been reached.

Changing motor speed provides a way to observe how the vibration indicator behaves under different operating settings. The CSV download facility supports offline examination of recorded readings and can be used to compare motor operation over time. However, the supplied project information does not include verified numerical test results, a completed experimental log, or calibrated vibration limits. Therefore, no numerical accuracy, response-time, fault-detection, or industrial reliability claims are made in this paper. Such values should be added only after testing the final assembled prototype.

A. Recommended Experimental Evaluation

The system should be tested at repeatable motor settings and under clearly documented mounting conditions. For each test, record the motor speed setting, sensor readings, vibration indicator, displayed state, buzzer response, and any relevant observation. Repeated measurements can be used to estimate the normal range and to check whether the warning and critical thresholds behave consistently. The dashboard and CSV file should also be checked to confirm that the recorded values agree with the live display.

Evaluation item	Value to record after testing
Sensor sampling interval	[Measured value] ms
Normal-state vibration range	[Measured range]
Warning threshold	[Configured value]
Critical threshold	[Configured value]
Dashboard update interval	[Measured value] ms
CSV logging interval	[Configured value] s
Buzzer activation response	[Observed / measured result]

X. LIMITATIONS

The current prototype uses predefined vibration thresholds rather than a trained machine-learning model. Thresholds may vary with motor speed, sensor mounting, mechanical structure, and environmental vibration. The MPU6050 is a low-cost sensor and its readings require appropriate calibration and interpretation. A resultant acceleration magnitude alone does not identify a particular fault type. More advanced analysis, such as frequency-domain examination and comparison against validated fault datasets, would be needed for more detailed diagnosis.

The BO motor is a small demonstration load and does not reproduce all operating conditions of industrial machinery. The web dashboard depends on the local Wi-Fi connection, and the quality of the displayed history depends on the sampling and logging implementation. The prototype requires additional long-duration testing, repeatability studies, and safety review before any industrial use.

XI. CONCLUSION

This paper presented an ESP32-based AI Preventive Maintenance System prototype for vibration-based machine condition monitoring. The system integrates an MPU6050 sensor, potentiometer-based motor speed control, L298N motor driver, BO DC motor, LCD, buzzer, local web dashboard, and CSV data logging. The controller calculates a vibration indicator and classifies the machine condition using predefined NORMAL, WARNING, and CRITICAL thresholds. The prototype demonstrates the practical integration of embedded sensing, motor control, local visualization, and alert generation. Its current threshold-based implementation provides a foundation for further development, while validated fault prediction and industrial deployment remain outside the demonstrated scope.

XII. FUTURE SCOPE

Future work may include collecting a larger vibration dataset under controlled operating conditions and using it to train and validate a machine-learning model for fault classification. Additional signal-processing methods, such as filtering and frequency-domain analysis, may help distinguish different vibration patterns. Thresholds may also be made speed-dependent after sufficient baseline measurements are collected. Other possible improvements include cloud data storage, mobile notifications, longer-term trend analysis, multiple-motor monitoring, and a more detailed dashboard with historical comparison and maintenance records. Each improvement should be evaluated on the final hardware for reliability, processing requirements, and practical usefulness.

ACKNOWLEDGMENT

The authors would like to acknowledge the Department of Electronics and Communication Engineering, Malla Reddy University, Hyderabad, and the project guide for their support and guidance.

REFERENCES

[1] Espressif Systems, “ESP32 Series Datasheet,” Espressif Systems.

[2] TDK InvenSense, “MPU-6000 and MPU-6050 Product Specification.”

[3] Arduino, “Arduino Documentation.” Available: https://docs.arduino.cc/

[4] STMicroelectronics, “L298 Dual Full-Bridge Driver Datasheet.”

[5] Espressif Systems, “ESP32 Programming Guide.” Available: https://docs.espressif.com/.